

Module 5 Week B — Core Skills Drill: Tree-Based Model Basics

Train a decision tree, extract feature importances, and train a random forest with balanced class weights. This drill targets retrieval practice — write the code from the reading’s concepts, not by copying snippets.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment:** [Accept the Drill 5B assignment](#)
2. Clone your repo and open it in VS Code.
3. The repo contains `data/telecom_churn.csv` — a subset of the Petra Telecom dataset with numeric and categorical columns and a binary `churned` target. The drill selects 8 numeric features (including `contract_months`, a numeric encoding of contract length) and splits the data in the `__main__` block of `drill.py` — review the block to see how `X_train`, `X_test`, `y_train`, `y_test` are created before your functions are called.
4. Install dependencies:

```
pip install -r requirements.txt
```

Tasks

Complete the functions in `drill.py`. The file contains function signatures with docstrings — you implement the logic. Run your code with `python drill.py` to verify outputs before pushing.

Task 1 – Train a Decision Tree

Implement `train_decision_tree(X_train, y_train)`:

- Train a `DecisionTreeClassifier` with `max_depth=5` and `random_state=42`
- Fit the model on the provided training data
- Return the fitted model

What the autograder checks: The returned object is a fitted `DecisionTreeClassifier` with `max_depth=5`.

Task 2 – Extract Feature Importances

Implement `get_feature_importances(model, feature_names)`:

- Extract `feature_importances_` from the trained decision tree
- Return a dictionary mapping each feature name to its importance value
- The dictionary must be sorted by importance in descending order

What the autograder checks: Returns a dictionary with the correct number of entries, values sum to approximately 1.0, and entries are sorted descending by value.

Task 3 – Train a Random Forest with Balanced Class Weights

Implement `train_balanced_forest(X_train, y_train, X_test, y_test)`:

- Train a `RandomForestClassifier` with `n_estimators=100`, `class_weight='balanced'`, and `random_state=42`
- Fit the model on the training data
- Compute precision, recall, and F1 score on the test set (use `precision_score`, `recall_score`, `f1_score` from `sklearn.metrics`)
- Return a dictionary with keys `"precision"`, `"recall"`, and `"f1"`

What the autograder checks: The model uses `class_weight='balanced'`; recall on the positive class exceeds the recall of a default (unweighted) random forest; precision, recall, and F1 are all present and greater than 0.

Submission

1. Create a branch named `drill-5b-tree-basics`.
2. Complete all three functions in `drill.py`.
3. Push your branch and open a PR to `main`.
4. Verify that the autograder passes (green check on the PR).
5. Paste your PR URL into TalentLMS → Module 5 Week B → Core Skills Drill 5B to submit this assignment.